Application Business Intelligence
Démissions d'un organisme bancaire

Mehdi Rayane

30 mars 2026

Résumé

Ce document présente l'**exploration**, le **nettoyage**, la **concaténation**, le **recodage**, le **prétraitement** et le **découpage** train / validation / test (`decoupage.py`). Objectif : **classification** du risque de démission. Sorties : `exploration/`, `nettoyage/`, `concatenation/`, `recodage/`, `pretraitement/`, `decoupage/`.

Table des matières

1 Contexte et objectifs **3**

2 Exploration **3**
2.1 Données et principes . . . 3
2.2 Outils mis en œuvre . . . 3
2.3 Organisation des sorties . . . 3
2.4 Résultats illustrés (sélection) . . . 3
2.4.1 Univarié . . . 3
2.4.2 Corrélations entre variables numériques . . . 3
2.4.3 Associations quanti–quanti (nuages de points) . . . 7
2.4.4 Quanti–quali : boîtes à moustaches . . . 7
2.5 Synthèse et limites de cette phase . . . 7

3 Nettoyage **7**
3.1 Ces données nécessitent-elles un nettoyage ? . . . 9
3.2 Faut-il écarter certaines instances qui ne sont pas liées au problème ? . . . 9
3.3 Sur quoi se base-t-on pour prendre cette décision ? . . . 10
3.4 Y a-t-il des valeurs manquantes ? . . . 10
3.5 Y a-t-il des valeurs aberrantes ? . . . 10
3.6 Y a-t-il des attributs redondants ? . . . 10
3.7 Y a-t-il des attributs superflus ? . . . 11
3.8 Les valeurs numériques correspondent-elles à des grandeurs métriques, ordinales ou catégorielles ? . . . 11
3.9 Comment traiter ces différents problèmes ? . . . 11

4 Concaténation **12**
4.1 Faut-il concaténer les tables ? . . . 12
4.2 Comment concaténer techniquement ? . . . 13
4.3 Résultats de la concaténation et lecture descriptive . . . 13
4.4 Choix pour la modélisation (distinct de la concaténation déjà réalisée) . . . 14

5 Recodage **14**
5.1 Principes 14
5.2 Tester l'effet du codage sur les outils 14

6 Prétraitement **15**
6.1 Réduction de dimension (ACP / PCA) 15
6.2 Sélection de variables (SelectKBest) 15
6.3 Comparaison des approches et paramètres 15

7 Découpage **16**
7.1 Rôles des trois jeux 16
7.2 Mise en œuvre (`decoupage.py`) 16
7.3 Précaution 16

8 Répartition du travail (phase Exploration) **16**

1 Contexte et objectifs

L'objectif du projet est d'identifier des profils de sociétaires à risque de démission, puis de construire ultérieurement un score d'attrition et d'en analyser l'explicabilité. La phase documentée ici se limite à l'**exploration descriptive et associative** des données historiques (`table1`) et de l'échantillon mélangé actuels/passés (`table2`), conformément au cahier des charges du module.

2 Exploration

2.1 Données et principes

— **Fichiers** : `data/table1.csv` (démissionnaires, 1999–2006) et `data/table2.csv` (échantillon aléatoire incluant démissionnaires et non-démissionnaires).

— **Aucun nettoyage** : les valeurs aberrantes ou sentinelle (ex. dates `0000-00-00`, `31/12/1900`) sont conservées telles quelles pour ne pas biaiser cette étape par des choix implicites.

— **Exclusion** : la colonne `ID` est exclue des analyses (pas de lien entre les deux tables).

— **Typage** : classification *quantitatif* vs *qualitatif* selon les types lus par `pandas` (entiers et flottants = quantitatifs ; texte = qualitatifs).

2.2 Outils mis en œuvre

L'exploration est automatisée par le script `exploration.py` à la racine du projet, qui s'appuie sur deux bibliothèques internes :

`analyse` Résumés univariés (moyenne, médiane, écart-type, quartiles, mode...) ; bivarié quanti–quanti (Pearson, Spearman, Kendall et tests associés) ; quali–quali (χ^2, degrés de liberté, Cramér V) ; quanti–quali (Kruskal–Wallis, ANOVA à un facteur, ε^2 pour Kruskal–Wallis).

`graphique` Histogrammes, diagrammes en barres, nuages de points, heatmaps de corrélation, boîtes à moustaches, barres de contingence.

Les dépendances sont listées dans `requirements.txt` (`pandas`, `numpy`, `scipy`, `matplotlib`, `seaborn`).

2.3 Organisation des sorties

Pour chaque table, les résultats sont séparés en deux sous-dossiers :

```
exploration/table1/univariate/ ;   exploration/table1/bivariate/
exploration/table2/univariate/ ;   exploration/table2/bivariate/
```

On y trouve des fichiers CSV récapitulatifs (`resume_univarie_*.csv`, `resume_bivarie_*.csv`) et l'ensemble des graphiques générés (toutes les paires de colonnes hors `ID` pour le bivarié, plus des heatmaps Pearson et Spearman sur les variables numériques).

2.4 Résultats illustrés (sélection)

2.4.1 Univarié

La figure 1 montre la distribution des revenus dans `table2` : forte asymétrie (nombreux zéros, queue droite). La figure 2 présente l'âge à l'adhésion dans `table1`.

2.4.2 Corrélations entre variables numériques

Les heatmaps (Pearson) synthétisent les associations linéaires entre toutes les colonnes numériques d'une même table (figures 3 et 4).

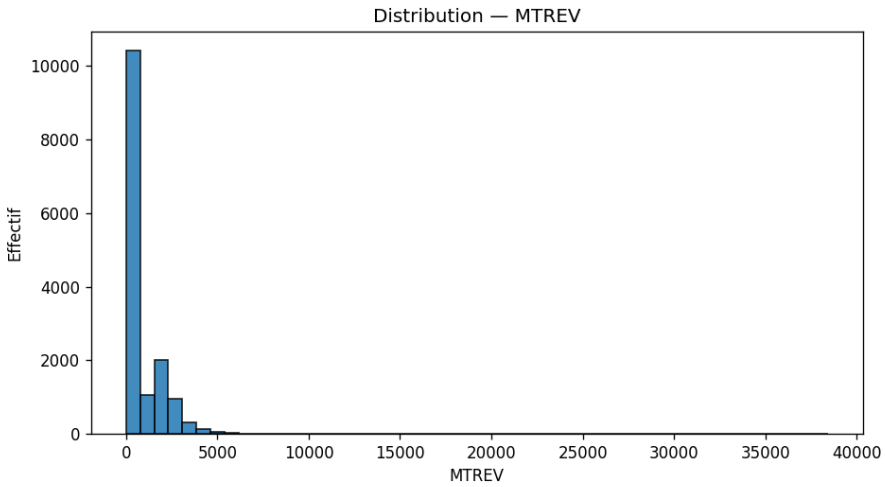

FIGURE 1 – `table2` — distribution de `MTREV` (montant des revenus).

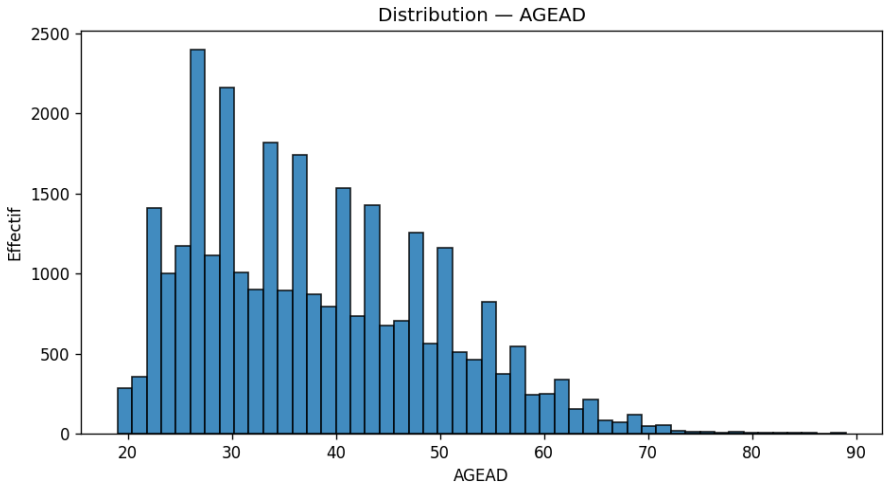

FIGURE 2 – `table1` — distribution de `AGEAD` (âge à l'adhésion).

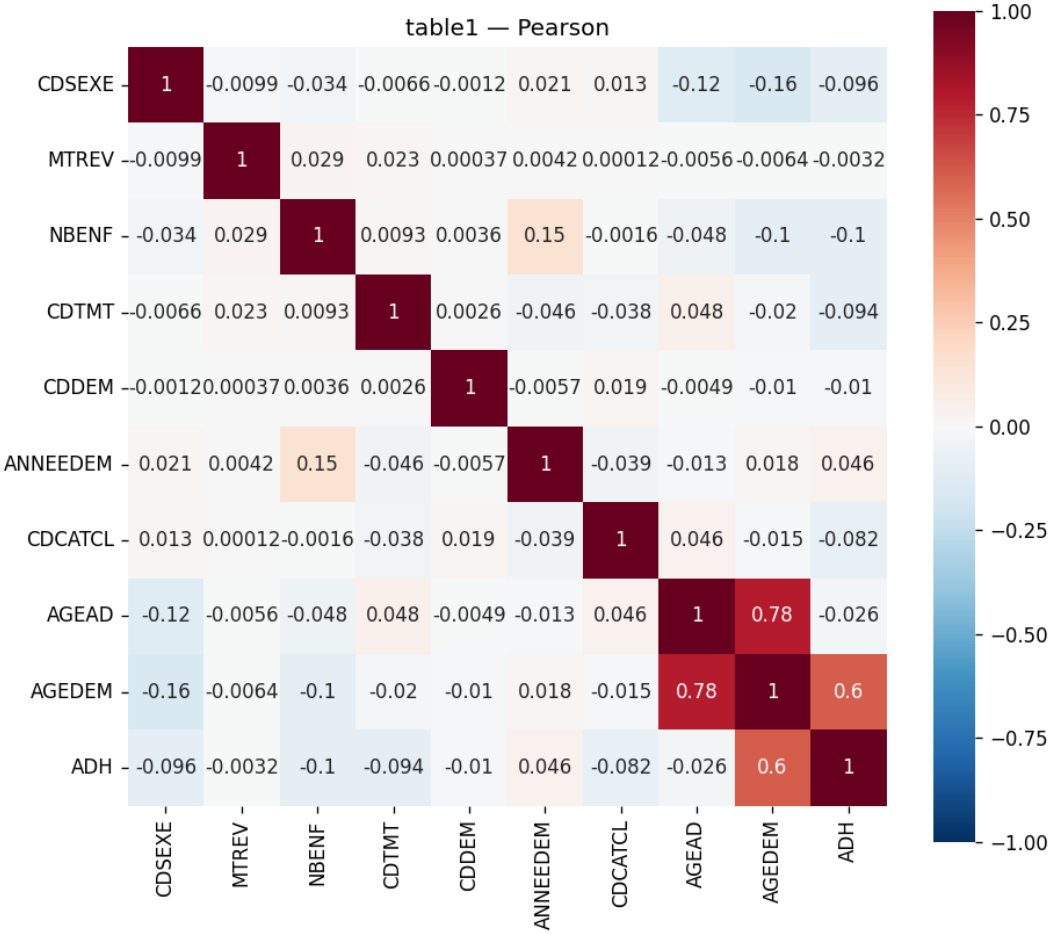

FIGURE 3 – `table1` — matrice de corrélations de Pearson entre variables quantitatives.

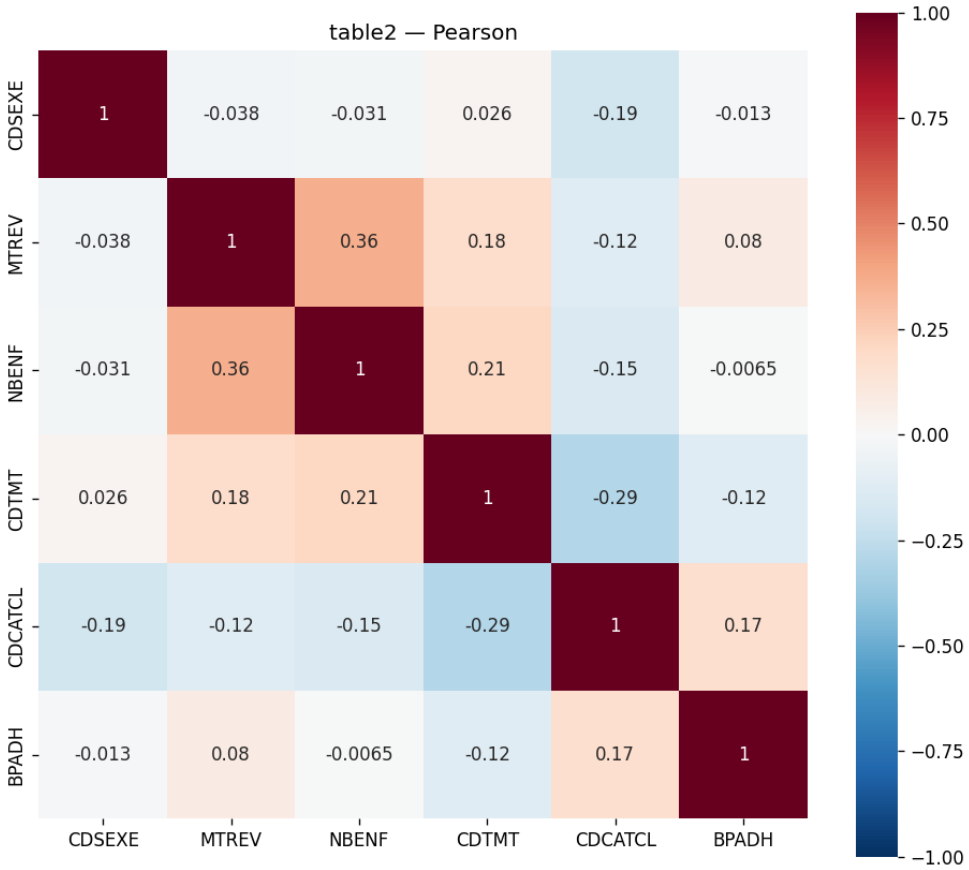

FIGURE 4 – `table2` — matrice de corrélations de Pearson entre variables quantitatives.

2.4.3 Associations quanti–quanti (nuages de points)

La figure 5 met en évidence une corrélation forte entre âge à l'adhésion et âge à la démission dans `table1`. La figure 6 relie l'ancienneté (`ADH`) à l'âge à la démission.

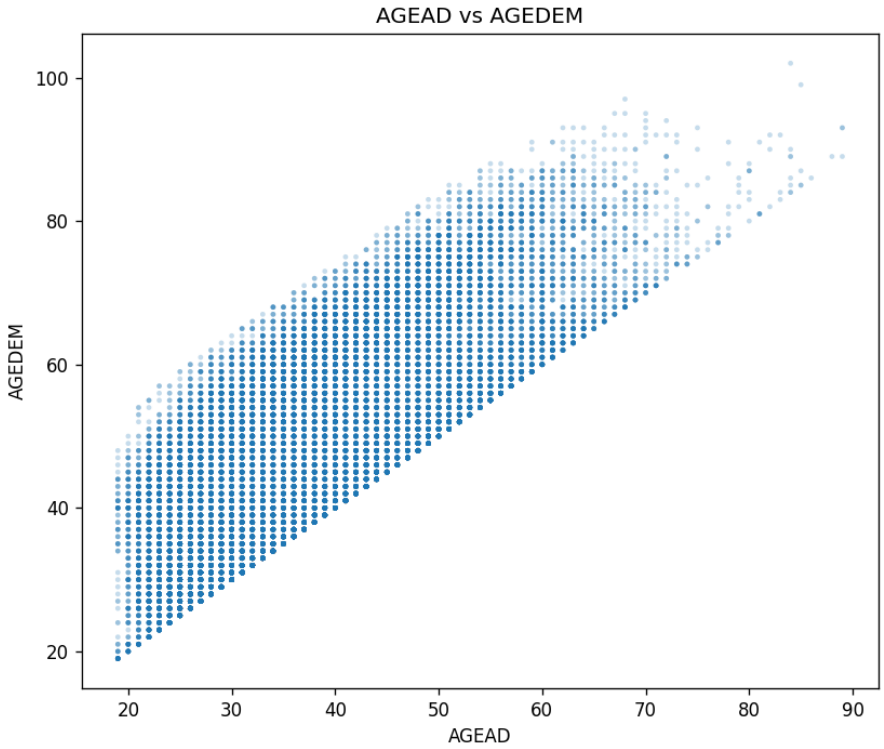

FIGURE 5 – `table1` — `AGEAD` vs `AGEDEM` (nuage de points).

Dans `table2`, le lien entre `MTREV` et `NBENF` est illustré figure 7.

2.4.4 Quanti–quali : boîtes à moustaches

La figure 8 compare la distribution de `AGEAD` selon les modalités de `RANGAGEAD` (tranche d'âge à l'adhésion) : la redondance informationnelle entre âge numérique et tranche est visible.

2.5 Synthèse et limites de cette phase

— Les tests sont nombreux (toutes les paires) : les p-valeurs doivent être interprétées avec prudence (multiplicité) ; les tailles d'effet (coefficients de corrélation, Cramér V, ε^2) complètent la lecture.

— Les variables à très nombreuses modalités (dates en chaîne de caractères) produisent des χ^2 élevés et des graphiques de contingence peu lisibles : elles seront à traiter ultérieurement (agrégation, features temporelles) hors périmètre « exploration brute ».

— La suite du projet (modélisation supervisée, score, explicabilité) s'appuiera sur une définition explicite de la cible démission à partir de `table2` (`DTDEM`, `CDMOTDEM`, etc.).

3 Nettoyage

L'objectif final du projet est d'**identifier les sociétaires à risque de départ**, ce qui sera réalisé ultérieurement par des **méthodes de classification** supervisées. La préparation des données n'est pas une simple suppression de lignes : il s'agit de rendre les variables **cohérentes**,

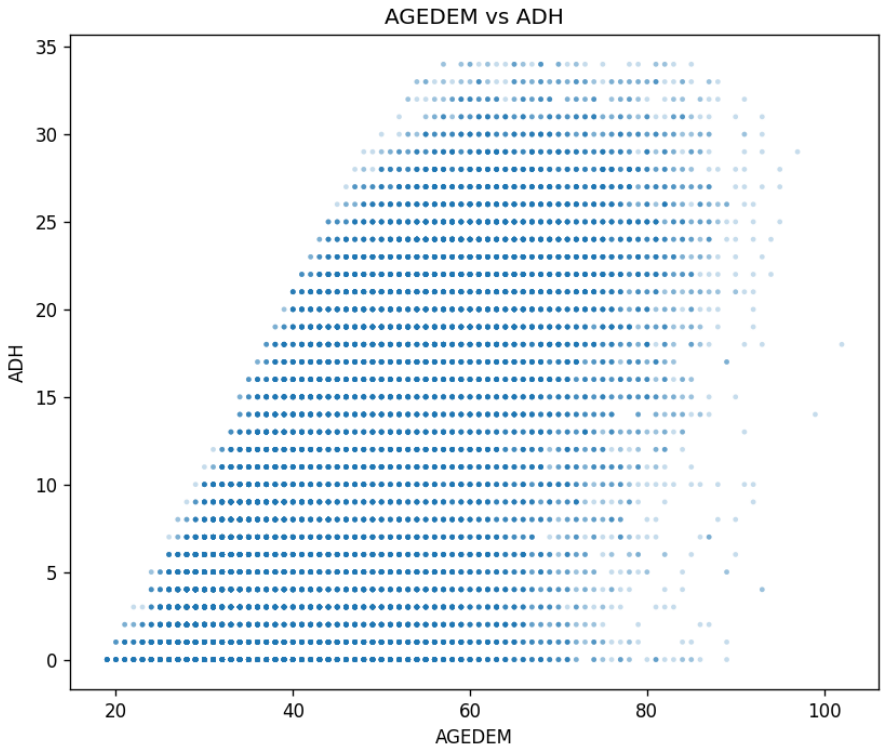

FIGURE 6 – `table1` — `ADH` (durée d'adhésion) vs `AGEDEM`.

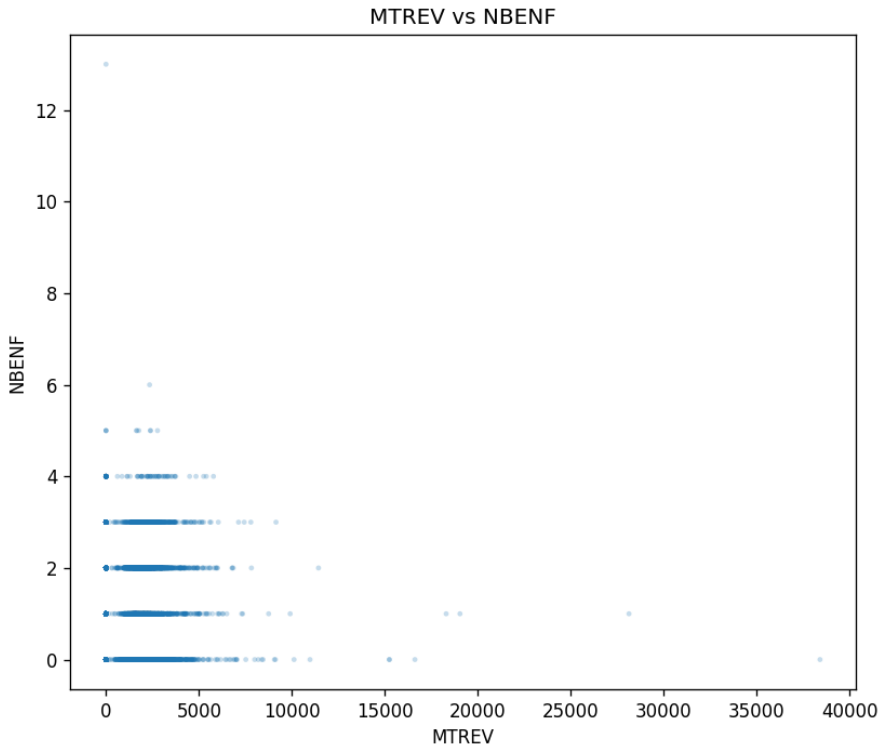

FIGURE 7 – `table2` — `MTREV` vs `NBENF`.

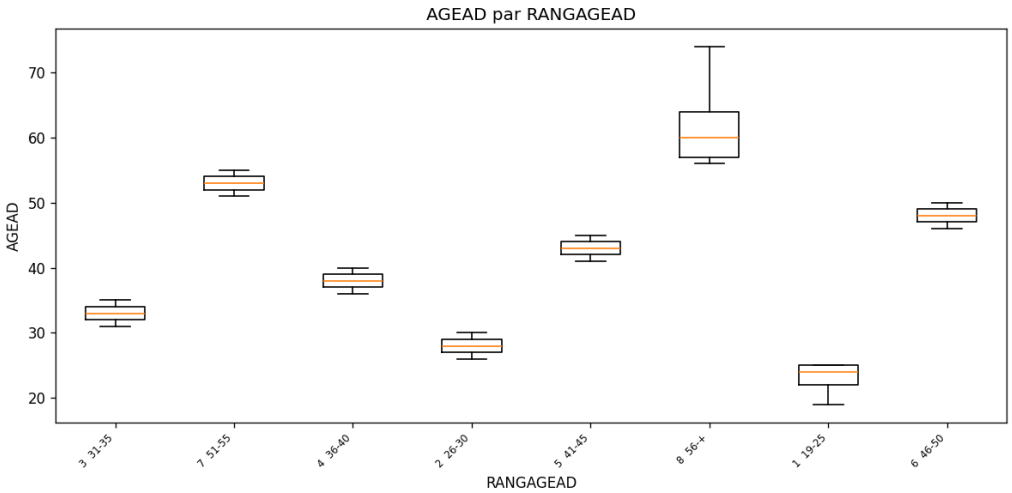

FIGURE 8 – `table1` — `AGEAD` par modalité de `RANGAGEAD`.

interprétables et **exploitables** par un modèle, tout en documentant les choix. Les constats ci-dessous s'appuient sur le script `nettoyage.py` (sorties dans `nettoyage/`) et sur l'exploration précédente.

3.1 Ces données nécessitent-elles un nettoyage ?

Constat. Les fichiers mélangent **dates en texte**, **codes sentinelles** explicites dans l'énoncé (`0000-00-00` pour une naissance inconnue, `31/12/1900` pour un non-démissionnaire dans `table2`), des **entiers** parfois catégoriels, et des distributions très **asymétriques** (ex. `MTREV`, figure 1).

Idée. Le « nettoyage » recouvre ici la **préparation** : recodage des sentinelles, parsing ou dérivation de features à partir des dates, encodage des modalités, transformations éventuelles des variables continues à queue lourde.

Solution. Mettre en place une chaîne de traitements **traçable** (reproductible) avant l'apprentissage : définition de la variable cible (démission oui/non), traitement des manquants structurels, sélection ou dérivation de variables pour limiter la redondance.

Explication. Sans préparation, un classifieur mélange absences d'information codées et valeurs réelles, et peut être dominé par quelques modalités extrêmes ou par des doublons informationnels.

3.2 Faut-il écarter certaines instances qui ne sont pas liées au problème ?

Constat. Le script `nettoyage.py` ne détecte **aucune ligne entièrement vide** sur `table2` (15 022 lignes). Les valeurs atypiques repérées relèvent surtout de **codes métier**, pas d'enregistrements invalides. Par ailleurs, `table1` ne contient que des **démissionnaires** et `table2` un mélange : ce sont des **populations différentes**.

Idée. « Non lié au problème » doit être défini par le **périmètre métier** (qui prédire ? à quelle date d'extraction ?), pas seulement par un critère statistique automatique.

Solution. Ne pas supprimer massivement les lignes portant des sentinelles ; éventuellement **restreindre la population** (ex. critères d'éligibilité) uniquement si le cahier des charges le fixe explicitement. Ne pas fusionner naïvement `table1` et `table2` pour un score sur sociétaires « actuels » sans stratégie d'échantillonnage.

Explication. Exclure des individus sans justification métier **biaise** l'inférence ; les sentinelles se **recodent** (manquant, indicateur, modalité dédiée).

3.3 Sur quoi se base-t-on pour prendre cette décision ?

Constat. Plus sources se combinent : dictionnaire des variables, fichiers `manquants_table*.csv`, `sentinelles_table*.json`, `redondances.csv`, `typologie_attributs.csv`, résultats de l'exploration (corrélations, Cramér V), et contraintes du futur modèle (ex. arbres vs régression).

Idée. Décision **data + objectif + modèle** : une règle purement automatique (seuil IQR, etc.) ne suffit pas sans validation métier.

Solution. Documenter chaque règle (recodage, exclusion éventuelle, transformation) dans le rapport de préparation et, plus tard, dans la validation du modèle.

Explication. La traçabilité est indispensable pour la **reproductibilité** et l'**explicabilité** attendues en BI.

3.4 Y a-t-il des valeurs manquantes ?

Constat. Table1 : seuls des manquants `pandas` sur `RANGADH` — 2 584 cas (8,52 % du fichier). **Table2** : `CDMOTDEM` compte 14 195 valeurs manquantes (94,49 %) ; les autres colonnes n'ont pas de NaN dans l'import brut.

Idée. Le volume élevé sur `CDMOTDEM` est **structurel** : le motif de démission n'existe que pour les démissions déclarées.

Solution. Pour `RANGADH` : imputation, modalité « inconnu », ou modèle gérant le NA. Pour `CDMOTDEM` : l'intégrer comme **non applicable** hors démission ; la cible de classification reposera plutôt sur `DTDEM` / absence de motif selon la définition opérationnelle de la démission.

Explication. Un taux de manquants élevé n'indique pas toujours une « mauvaise qualité globale » : il peut refléter la **sémantique** de la variable.

3.5 Y a-t-il des valeurs aberrantes ?

Constat. Deux types coexistent. (1) **Sentinelles** documentées : `DTNAIS` = `0000-00-00` (164 lignes dans `table2`) ; `DTDEM` = `31/12/1900` pour marquer les **non-démissionnaires** (14 474 lignes). (2) **Extrêmes numériques** : par exemple `MTREV` jusqu'à 1 524 490 (table1) et 38 413 (table2) ; la règle IQR (fichiers `outliers_iqr_*.csv`) classe une part notable de valeurs comme « hors boîte » pour `MTREV` et `NBENF`, avec prudence pour les variables à masse en zéro.

Idée. Distinguer **code métier** (à recoder) et **valeur extrême plausible** (revenu élevé, famille nombreuse).

Solution. Recoder les sentinelles ; pour les continuums, envisager **log(MTREV+1)**, winsorisation, ou modèles peu sensibles aux queues (arbres) sans suppression systématique des extrêmes.

Explication. Supprimer agressivement les gros revenus peut **casser** la représentation des profils à risque réels.

3.6 Y a-t-il des attributs redondants ?

Constat. L'exploration montre une forte dépendance entre **âges numériques** (`AGEAD`, `AGEDEM`) et les **tranches** associées (`RANGAGEAD`, `RANGAGEDEM`), entre **`ADH`** et **`RANGADH`**, et une information temporelle dupliquée entre **`ANNEEDEM`**, **`DTDEM`** et **`RANGDEM`** (voir aussi figure 8).

Idée. La redondance peut **gonfler** la variance des estimateurs ou favoriser le sur-ajustement selon le modèle.

Solution. Ne conserver qu'**une représentation par famille** (âge continu *ou* tranche ; une échelle de temps cohérente pour la démission).

Explication. Mieux vaut une seule variable bien définie que trois colinéaires.

3.7 Y a-t-il des attributs superflus ?

Constat. L'identifiant `ID` n'apporte pas d'information prédictive. Le champ `BPADH` reste **non documenté** dans l'énoncé. Sur `table1`, le code `CDDEM` est **quasi constant** : la figure 9 montre que l'effectif se concentre sur une seule modalité (valeur 2), ce qui confère une **variance nulle ou négligeable** à cet attribut pour la discrimination.

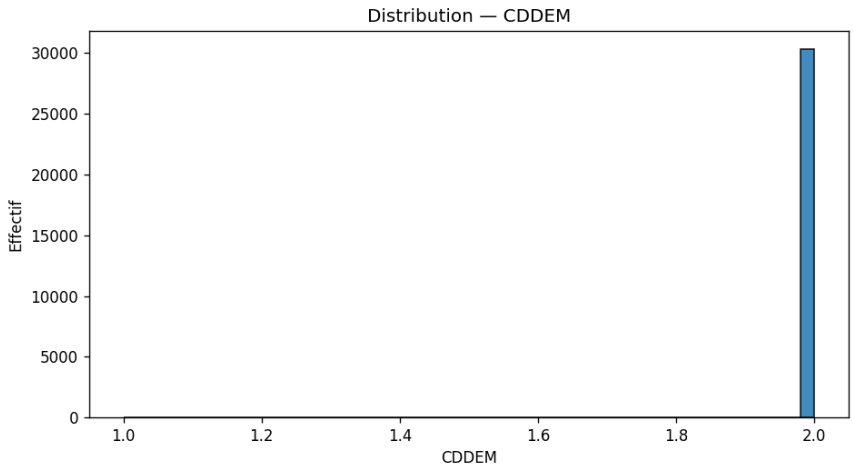

FIGURE 9 – `table1` — distribution de `CDDEM` : quasi-totalité des observations sur une modalité (variance utile nulle pour un classifieur).

Idée. Un prédicteur constant ou quasi constant ne contribue pas au score d'attrition ; il peut être retiré ou ignoré dans la sélection de variables.

Solution. Exclure `ID` des features ; traiter `BPADH` après clarification métier ; retirer `CDDEM` de la liste des prédicteurs pour `table1` si le constat de quasi-constance se confirme sur le périmètre d'apprentissage.

Explication. Pour la classification, l'objectif est le **pouvoir discriminant** vis-à-vis de la cible ; les variables sans variation n'en ont pas.

3.8 Les valeurs numériques correspondent-elles à des grandeurs métriques, ordinales ou catégorielles ?

Constat. Le typage `pandas` (entier vs texte) est trompeur : `CDSEXE`, `CDTMT`, `CDCATCL` sont des **codes catégoriels** ; les `RANG*` sont des **ordinaux** (tranches) ; `AGEAD`, `AGEDEM`, `ADH`, `MTREV` se rapprochent de variables **métriques** (continues ou comptages).

Idée. Le niveau de mesure guide l'**encodage** (one-hot, ordinal respectueux, conservation en nombre) et l'interprétation des coefficients ou importances.

Solution. Encoder les nominaux sans ordre naturel ; traiter les ordinaux avec prudence ; ne pas interpréter linéairement un code sexe ou statut comme une « grandeur » sans validation.

Explication. Un modèle de classification **correctement alimenté** évite les faux continuums et améliore la lisibilité des résultats pour la campagne CRM.

3.9 Comment traiter ces différents problèmes ?

Constat. Les problèmes se cumulent (manquants, sentinelles, extrêmes, redondance, typage).

Idée. Enchaîner les traitements dans un ordre logique : **cible** → **recodage** → **sélection/dérivation de variables** → **encodage** → **échantillon d'apprentissement/validation**.

Solution. (1) Définir la cible binaire (démission) à partir de `DTDEM` et/ou `CDMOTDEM`. (2) Convertir les sentinelles en NA ou indicateurs. (3) Parser les dates ou en extraire l'ancienneté et l'âge à la date d'extraction (2007). (4) Réduire la redondance (âge vs tranches). (5) Transformer ou

binariser `MTREV` si nécessaire. (6) Imputer ou coder le manquant lorsque pertinent. (7) Entraîner le classifieur et valider hors échantillon.

Explication. Cette chaîne relie directement le diagnostic (`nettoyage/`) à l'**objectif métier** : prioriser les sociétaires à risque pour une action de rétention.

4 Concaténation

Les données sont réparties entre **`table1`** (démissionnaires historiques, 1999–2006) et **`table2`** (échantillon aléatoire de sociétaires actuels et passés). L'énoncé précise que les **identifiants `ID` ne correspondent pas** d'une table à l'autre : il n'existe donc **pas de jointure** possible sur la clé `ID`. Le script `concatenation.py` implémente l'**empilement vertical** sur les colonnes communes, avec sorties dans `concatenation/` : `resume_strategies.json`, `echantillon_union_500lignes.csv`, `descriptifs_variables_communes.csv` (moyennes/médianes indicatives), `synthese_concatenation.txt`.

À ne pas confondre. Il y a **deux questions différentes** :

1. **Concaténer techniquement les fichiers ? — Oui, c'est fait** : les lignes de `table1` et `table2` sont empilées sur les dix colonnes communes, avec `source` et de nouveaux `ID` (section 4.2 ci-dessous). Cela produit une **table unique** exploitable pour exploration ou pour des modèles qui **utilisent explicitement** l'origine (`source`) ou une pondération.
2. **Sur quelle base entraîner surtout le classifieur churn / non-churn ?** — Là, on **recommande de partir de `table2`** comme **jeu principal**, parce qu'y coexistent démissionnaires et non-démissionnaires dans **un même échantillon** ; la cible se déduit de `DTDEM`. Ce n'est **pas** dire « on n'a pas concaténé » : c'est dire qu'**entraîner uniquement sur l'union sans précaution** mélange une population (historique **tous** démissionnaires) avec une autre et **déforme** la proportion de positifs par rapport à ce qu'on observe dans `table2`.

4.1 Faut-il concaténer les tables ?

Constat. Les deux tables ne décrivent pas la même **population statistique** : `table1` ne contient que des **démissionnaires** ; `table2` contient à la fois des démissionnaires et des non-démissionnaires (date de démission sentinelle `31/12/1900`). Les colonnes se **recouvrent partiellement** : **dix** attributs communs (`ID`, `CDSEXE`, `MTREV`, `NBENF`, `CDSITFAM`, `DTADH`, `CDTMT`, `CDMOTDEM`, `CDCATCL`, `DTDEM`), alors que neuf attributs sont propres à `table1` (âges, tranches, `ANNEEDEM`, etc.) et deux à `table2` (`DTNAIS`, `BPADH`).

Idée. Concaténer n'est pas une nécessité logique : c'est un choix de **modélisation**. On peut travailler sur `table2` seul pour un problème supervisé **churn / non-churn**, ou fusionner des informations sous conditions strictes.

Solution. Trois approches typiques :

1. **Union verticale** sur les colonnes communes uniquement, avec une variable `source` $\in$ {`table1`, `table2`} et un réidentifiant les `ID` locaux pour éviter les collisions entre fichiers ;
2. **`table2` seul** : base naturelle pour estimer la probabilité de démission observée dans l'échantillon, avec cible dérivée de `DTDEM` ;
3. **Union + label explicite** : étiqueter chaque ligne (ex. démission oui/non) en traitant `table1` comme **tous positifs** historiques et `table2` selon la sentinelle — utile pour diagnostiquer le **déséquilibre** induit par la fusion.

Explication. Concaténer « pour avoir plus de lignes » sans tenir compte du **biais de sélection** (historique démissionnaires vs mélange) peut **fausser** un classifieur et sur-représenter la classe « démission ».

4.2 Comment concaténer techniquement ?

Constat. Sans clé métier commune, seule la **concaténation verticale** (`pandas.concat` sur l'axe des lignes) sur un sous-ensemble de colonnes **harmonisées** est cohérente. Les colonnes absentes d'un côté peuvent être laissées vides ou exclues ; ici l'union se limite aux **dix** colonnes communes pour éviter des trous massifs.

Idée. Dupliquer les `ID` bruts après empilement serait source d'ambiguïté : deux individus différents pourraient porter le même numéro. D'où la création d'un identifiant global ou l'exclusion de `ID` des analyses.

Solution. Choix retenu et justification. On concatène **uniquement** sur les **dix** attributs communs, car ce sont les seuls alignables sans imputation arbitraire entre fichiers. On ajoute **`source`** pour savoir si la ligne provient de l'historique démissionnaires ou de l'échantillon mixte. On régénère des **`ID` uniques** dans l'union et on conserve l'identifiant d'origine en `id_fichier`, afin d'éviter qu'un même numéro ne désigne deux personnes différentes selon la table. Un extrait (`echantillon_union_500lignes.csv`) est exporté dans `concatenation/` pour contrôle, sans dupliquer tout le jeu dans le dépôt.

Explication. Cette procédure est une **union ensembliste** de lignes, **pas** une jointure relationnelle : elle ne restitue pas un individu suivi dans les deux bases.

4.3 Résultats de la concaténation et lecture descriptive

Constat. Après exécution de `concatenation.py` : effectifs `table1` = 30 332, `table2` = 15 022, **union** = 45 354 lignes. Sur `table2`, avec démission si `DTDEM`≠`31/12/1900` : **548** démissions (**3,65 %**), **14 474** non-démissions. Si l'on étiquette naïvement l'union (toutes les lignes `table1` comme « démission » et `table2` selon la sentinelle), on obtient **30 880** lignes en classe démission et **14 474** en non-démission — la part de positifs **augmente fortement** par rapport au seul `table2`.

Idée. Le fichier `descriptifs_variables_communes.csv` donne, pour chaque variable **numérique** commune, moyenne et médiane **séparément** dans `table1` et `table2`. Ce n'est **pas** un test d'hypothèse : ce sont des **indicateurs descriptifs** pour lecture exploratoire.

Variable	Méd. t1	Méd. t2	Moy. t1	Moy. t2
CDCATCL	21	10	21,00	13,56
CDSEXE	3	3	2,68	2,66
CDTMT	0	2	0,10	1,22
MTREV	0	0	289,00	681,73
NBENF	0	0	0,20	0,45

Table 1 – Lecture descriptive uniquement (fichier généré par `concatenation.py`, sans test statistique). Les écarts suggèrent des profils différents, à interpréter avec prudence.

Mise en garde (sans test du type Mann-Whitney). Sans procédure de **comparaison de distributions** (test non paramétrique, χ^2 pour le qualitatif, etc.), on ne peut **pas** conclure formellement que « les deux tables viennent de la même loi » ou l'inverse. On peut en revanche **tromper** un modèle en traitant l'union comme un échantillon homogène alors que `table1` est **restreint aux démissionnaires** : le biais de composition reste entier. Un test du type Mann-Whitney (ou autre) serait une piste pour **étayer** statistiquement une différence de profils ; ici seuls les descriptifs et le raisonnement métier sont mobilisés.

Explication. La concaténation technique reste **légitime** pour disposer d'un fichier unique sur variables communes ; l'interprétation pour la **classification** (pondération, variable `source`, jeu principal sur `table2`) demeure distincte.

4.4 Choix pour la modélisation (distinct de la concaténation déjà réalisée)

Constat. La **concaténation des fichiers** (union sur dix colonnes) est une **réalisation technique** documentée ci-dessus ; elle n'est pas annulée par la suite. En revanche, pour l'objectif **identifier les sociétaires en risque de départ** par **classification**, la **base la plus directe pour l'apprentissage supervisé** est `table2` : la **cible** y est dérivable de `DTDEM` / `CDMOTDEM` et les **deux classes** coexistent. `table1` reste un **historique** riche (tous démissionnaires), utile en analyse descriptive ou en complément (pondération, variable `source`, transfert), mais pas substitut logique de `table2` comme **seul** référent de prévalence churn/non-churn.

Idée. Utiliser l'**union** comme jeu d'entraînement **brut** sans tenir compte de `source` revient à mélanger un univers « 100 % démissionnaires » (`table1`) avec un mélange réel (`table2`) : le modèle voit une proportion de positifs **artificiellement élevée** par rapport à l'échantillon `table2`.

Solution. Recommandation pour le modèle : jeu d'apprentissage **principal** = `table2` (après préparation). **L'union produite par `concatenation.py`** sert de **vue unifiée** et peut alimenter d'autres usages (exploration, modèles avec `source`, repondération). Ce n'est pas une interdiction de concaténer — c'est un **choix de stratégie d'apprentissage**.

Explication. On sépare ainsi **mise en forme des données** (concaténation faite) et **stratégie de scoring** (privilégier `table2` pour coller à la population et aux deux classes observées ensemble).

5 Recodage

Pour la **fouille de données** et les algorithmes de classification (SVM, naïf bayésien, arbres, régression logistique, etc.), le **codage** des attributs doit être **compatible** avec l'outil : certaines méthodes exigent des nombres bornés ou normalisés, d'autres tolèrent des catégories brutes une fois encodées. Le script `recodage.py` (sorties dans `recodage/`) part de **`table2`** uniquement, conformément à la section précédente.

5.1 Principes

Constat. Les données brutes mélangent types (dates en texte, sentinelles, entiers catégoriels, revenus très asymétriques).

Idée. Appliquer, de façon documentée : **cible binaire** (`cible_churn`) ; **dérivation** (âge à 2007, ancienneté d'adhésion) ; **discrétisation** (quantiles sur `MTREV`) ; **normalisation** (deux variantes : Z-score et min–max sur le bloc numérique) ; **one-hot** sur les qualitatifs (`CDSITFAM`, `CDMOTDEM` avec modalité pour manquant).

Solution. Fichiers produits : `table2_recodage_brut_derive.csv` (variables dérivées + cible), `table2_matrice_modele.csv` (identifiant, cible, colonnes `z_*` et `mm_*`, indicatrices), `echantillon_matrice_modele_300lignes.csv`, `schema_recodage.json`, `synthese_recodage.txt`. Après exécution de `recodage.py` sur les données actuelles, la matrice modèle compte **15 022** lignes et **35** colonnes (dont `ID` et `cible_churn`) — les dimensions exactes sont aussi dans `schema_recodage.json`.

Explication. La normalisation est surtout utile pour les méthodes **à distance** (SVM à noyau, k-plus proches voisins) ou **gradient** ; les **arbres** sont souvent peu sensibles au centrage-réduction. Le one-hot évite d'imposer un ordre faux aux codes catégories. La date de référence pour l'âge et l'ancienneté est le **1er janvier 2007** (énoncé).

5.2 Tester l'effet du codage sur les outils

Constat. Le sujet recommande de **tester** l'effet du recodage selon l'algorithme retenu.

Idée. Construire au minimum **deux jeux** à partir du même échantillon : par ex. matrice avec seulement `z_*` vs. matrice avec `mm_*` vs. variables brutes discrétisées pour arbres ; comparer

AUC ou **taux de bon classement** sur une base test.

Solution. Plan d'expérience minimal : même partition train/test, même cible, en ne faisant varier que le pipeline de recodage ; consigner les résultats dans un tableau par (outil, codage).

Explication. Sans cette comparaison, on ne sait pas si un échec vient du **modèle** ou du **prétraitement**.

6 Prétraitement

Le **prétraitement** s'applique **après** le recodage, sur la matrice numérique produite (`recodage/table2_matri` Il vise notamment à **réduire la dimension** (moins de colonnes à traiter, temps de calcul parfois réduit) ou à **sélectionner** les variables les plus informatives pour la cible — au prix, pour certaines méthodes, d'une **interprétabilité** plus difficile. Le script `pretraitement.py` écrit les résultats dans `pretraitement/` ; il doit être exécuté **après** `recodage.py`.

6.1 Réduction de dimension (ACP / PCA)

Constat. La matrice recodée compte **33** attributs prédictifs (hors `ID` et cible). Pour des jeux plus grands, la dimension peut alourdir l'entraînement.

Idée. L'**Analyse en Composantes Principales** (PCA) projette les individus sur un nombre réduit d'**axes orthogonaux** (composantes), combinaisons linéaires des variables d'origine.

Solution. Deux variantes sont exportées : PCA conservant environ **95 %** de la variance (ici **neuf** composantes, variance cumulée $\approx 0{,}95$) ; PCA à **dix** composantes fixes (variance cumulée $\approx 0{,}97$). Fichiers : `matrice_pca_95var.csv`, `matrice_pca_10composantes.csv`, `pca_variance_expliquee.csv`, `pca_loadings_95var.csv` (contributions des variables initiales aux axes), `schema_pretraitement.json` (valeurs exactes après exécution), `synthese_pretraitement.txt`.

Explication / limite. Les composantes **ne** correspondent **pas** à des variables métier isolées : l'explicabilité pour le CRM (comprendre l'effet du revenu, de la situation familiale, etc.) est en général **plus difficile** qu'avec des variables brutes ou sélectionnées par nom. La PCA peut en revanche **décorréler** et stabiliser numériquement les entrées pour certains algorithmes.

6.2 Sélection de variables (SelectKBest)

Constat. On peut réduire la dimension en **gardant** un sous-ensemble de colonnes d'origine plutôt qu'en les mélangeant.

Idée. Test **F** d'ANOVA univarié entre les deux groupes de la cible (`SelectKBest`, $k = 10$) : classe les variables selon leur lien marginal avec `cible_churn`.

Solution. Fichiers `matrice_selectkbest_f10.csv` et `selectkbest_scores.csv` (scores et indicateur de sélection).

Explication. Les **noms** des variables restent lisibles, ce qui aide l'interprétation ; en revanche la sélection est **univariée** (ignore les interactions) et dépend de k.

6.3 Comparaison des approches et paramètres

Constat. Le sujet invite à **tester** plusieurs méthodes et paramétrages (nombre de composantes PCA, seuil de variance, k pour la sélection).

Idée. Même protocole expérimental que pour le recodage : **même** découpage train/test, **même** algorithme de fouille, en ne faisant varier que le prétraitement ; comparer métrique (AUC, précision, temps).

Solution. Consigner les résultats dans un tableau ; choisir le compromis **performance / temps / lisibilité** pour le rapport final.

Explication. Un bon score avec PCA n'implique pas que les axes soient **actionnables** pour la campagne de rétention sans analyse complémentaire (loadings, profils).

7 Découpage

Après recodage (et éventuel prétraitement), les données sont réparties en **trois** sous-ensembles disjoints. Le script `decoupage.py` part de `recodage/table2_matrice_modele.csv` et écrit les fichiers dans `decoupage/`.

7.1 Rôles des trois jeux

Constat. L'apprentissage automatique supervisé exige de mesurer la généralisation sans **biaiser** l'estimation par une utilisation multiple des mêmes lignes.

Idée. Distinguer :

— **Apprentissage (train)** : ajuster les paramètres du modèle (poids, règles, etc.) ;

— **Validation** : choisir les **hyperparamètres** (profondeur d'arbre, C pour un SVM, k pour k-NN, etc.) et **comparer** plusieurs algorithmes ou prétraitements ;

— **Test** : une seule estimation finale de la performance sur des données **jamais** utilisées pour l'entraînement ni pour le choix du modèle.

Solution. Ne consulter le **jeu test** qu'**après** fixation du modèle retenu (sinon risque de sur-ajustement au test).

Explication. Mélanger validation et test revient à **sur-estimer** la qualité sur données nouvelles.

7.2 Mise en œuvre (`decoupage.py`)

Constat. La matrice compte **15 022** lignes ; la cible `cible_churn` est déséquilibrée ($\approx 3{,}6\,\%$ de démissions).

Idée. Découpage **stratifié** sur `cible_churn` pour conserver une prévalence comparable entre jeux. Proportions retenues : **60 %** apprentissage, **20 %** validation, **20 %** test (d'abord 20 % mis de côté pour le test, puis 25 % du reste pour la validation).

Solution. Fichiers générés : `jeu_apprentissage.csv`, `jeu_validation.csv`, `jeu_test.csv`, `schema_decoupage.json`, `synthese_decoupage.txt`. Effectifs observés après exécution : **9 012** / **3 005** / **3 005** (arrondis cohérents avec 60/20/20 %).

Explication. Le `random_state` fixe la reproductibilité ; toute autre graine changerait les lignes mais pas la logique.

7.3 Précaution

Constat. Toute statistique (moyenne, écart-type, PCA...) calculée sur **tout** le fichier avant découpage puis appliquée aux trois jeux peut provoquer une **fuite d'information** vers le test.

Idée. Idéalement, **ajuster** les transformateurs (scaler, PCA, etc.) sur le **train** seulement, puis les appliquer à la validation et au test.

Solution. Pour ce projet, le recodage a été calculé sur l'ensemble `table2` : c'est une limite connue ; pour un protocole strict, recoder à partir du train ou utiliser une validation croisée sur le train.

Explication. Le rapport distingue **principe** (fit des prétraitements sur le train) et **implémentation actuelle** (matrice globale puis découpage).

8 Répartition du travail (phase Exploration)

Le tableau 2 décrit de façon équitable les apports respectifs pour la phase d'exploration documentée dans ce rapport. Les deux contributeurs ont participé au cadrage ; les tâches ont été réparties selon les compétences et la disponibilité.

Personne	Contribution (Exploration)
Mehdi	Cadrage méthodologique aligné sur le cours (analyse uni/bivariée, choix des familles de tests) ; relecture de l'énoncé et des dictionnaires de variables ; arbitrage sur les figures à retenir pour le rapport ; validation des interprétations (redondance âge/tranches, sens des sentinelles de dates).
Rayane	Implémentation Python : packages `analyse` et `graphique`, scripts `exploration.py`, `nettoyage.py`, `concatenation.py`, `recodage.py`, `pretraitement.py`, `decoupage.py` ; figures `latex/img` ; rédaction LaTeX (Exploration à Découpage).

Table 2 – Répartition indicative des tâches — phase Exploration uniquement.

Conclusion (jusqu'au découpage)

L'exploration et le nettoyage décrivent les données ; la concaténation unifie les tables ; le **recodage** construit la matrice modèle ; le **prétraitement** propose PCA et sélection de variables ; le **découpage** (`decoupage.py`) fournit les jeux **apprentissage**, **validation** et **test** pour entraîner les modèles, régler les hyperparamètres et estimer la généralisation. Il reste à **implémenter** les algorithmes de fouille et à respecter le protocole (ne pas « toucher » au test avant la fin).